

LAB 1: Message Class

Date: 16 June of 2025

Class Assignment :

```
//class declaration
```

```
class packet;
```

```
//class properties
```

```
bit [31:0] addr;
```

```
bit [31:0] data;
```

```
endclass
```

```
//declare the handle
```

```
packet pkt_1;
```

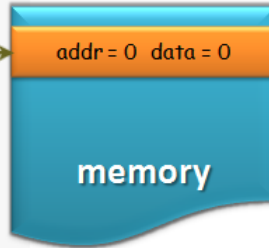
```
packet pkt_2;
```

```
//construct the object
```

```
pkt_1 = new();
```

```
pkt_2 = pkt_1; //assignment
```

0x1234

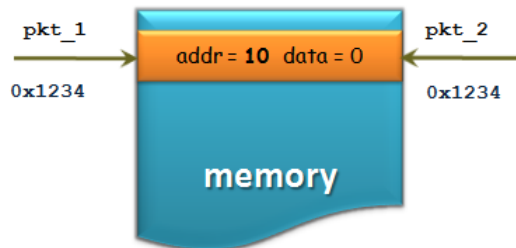


0x1234

With the assignment, both the objects will point the to same memory.

```
//changing the value of addr using pkt_2 handle
```

```
pkt_2.addr = 10;
```



Proffesor: Dr. Miguel Angel Rivera Acosta

Students: Luis Alberto Castañeda Montaño

José Antonio García Madrigal

Cesar Eduardo Inda Cenicerros

Alonso Emmanuel López Macías

Team: 9

Introduction

The main objective of this laboratory was to introduce the fundamental concepts of object-oriented programming within the context of verification in HDL (Hardware Description Language). Two classes were implemented: message class and message_class2. The first contains a string property and methods to initialize and display the message. The second class extends the first, inheriting its properties and methods while incorporating an additional attribute. This structure allowed for the exploration of key concepts such as inheritance, constructors (new()), encapsulation, and object handling.

Through the testbench module (tb), an object of the derived class was instantiated, its properties were modified, and methods were used to display the output in the simulation console. This reinforced the understanding of object-oriented modeling in System Verilog, an essential tool for developing advanced verification environments in modern digital design.

The code developed in this practice demonstrates how System Verilog, extends the capabilities of traditional hardware description languages by incorporating structures such as classes, inheritance, and methods. These features are crucial for creating reusable, scalable, and well-organized verification environments. Mastering these concepts not only facilitates working with UVM (Universal Verification Methodology) environments but also provides a broader perspective on object-oriented design, which is highly valued in the semiconductor industry.

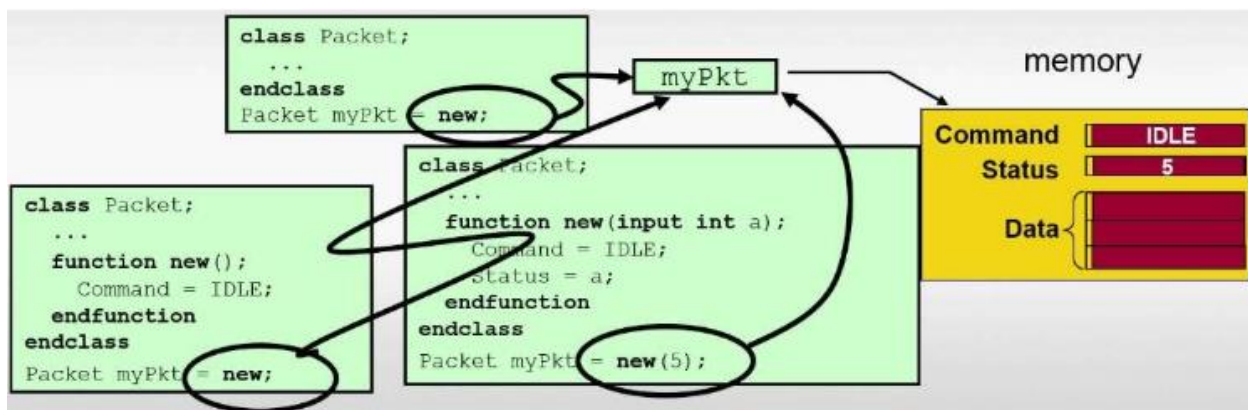


Figure: 1 SystemVerilog_Class

Requirements

- Compile & run the program in “**Vivado**”.

Development

```
#!/ Course: Certificación en diseño de circuitos integrados digitales
#!/ Client: COCYTEN 2024
#!/todo Owner: Team9_
/** Laboratory: message_class_sv

// This is a class, which is a user-defined data type
class message_class;

    string message; // Class property

    // The new method is the constructor of the class in SystemVerilog
    // The constructor method is used to create an object of this class type
    function new();
    |   message = "Hello world!";
    endfunction

    // Method to print the message member
    function void print_message();
    |   $display("%S", this.message);
    endfunction

endclass

// message_class2 is extended from the message_class, therefore
// this class (message_class2) inherits all the members from the parent class (message_class)
class message_class2 extends message_class;

    string message2; // Class property

    // The new method is the constructor of the class in SystemVerilog
    // The constructor method is used to create an object of this class type
    function new();
    |   super.new(); // Invokes the message_class constructor method
    |   message2 = "Good bye!";
    endfunction

    // Method to print the message2 member
    // This method also calls the print_message method inherited from parent class
    function void print_message2();
    |   this.print_message(); // Invokes the inherited print_message method
    |   $display("%S", this.message2);
    endfunction

endclass
```

Figure: 2 Message_Class

The consideration for the “*testbench*” was as following:

```
module tb;

    // Creates a handler for a variable called printer_, of message_class2 type
    message_class2 printer_;

    initial begin
        // Creates an object of message_class2 type, and links the object printer_ handler using the constructor
        printer_ = new();

        // Invokes the print_message2 method of printer_ object for printing the messages members
        printer_.print_message2();

        // Assign a new value to message member of printer_ object
        printer_.message = "Hola Mundo!";

        // Assign a new value to message2 member of printer_ object
        printer_.message2 = "Adios!";

        // Print the message member of printer_ object
        $display("%s", printer_.message);

        // Print the message2 member of printer_ object
        $display("%s", printer_.message2);

        // Finishes the simulation
        $finish;
    end

endmodule
```

Figure: 3 Testbench_MessageClass

Vivado

For the purpose of this practice, we chose the **ZCU104** in order to generate the simulation and project components through the selection of boards.

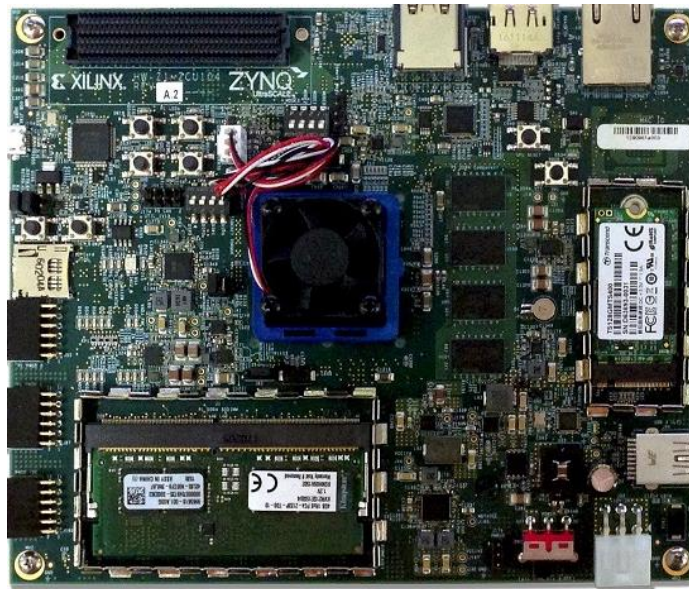


Figure: 4 ZCU104_Board

We create 2 folders one of them:

documentation → for add the information relevant in the laboratory.

rtl → for main design.

dv → for testbench's & simulations

TestBench

In the **"Tcl console"** was possible see the messages:

```
WARNING: Simulation object /tb/printer_ was not traceable in the design for the following reason:
Vivado Simulator does not support tracing of System Verilog Dynamic Type object.
INFO: [Wavedata 42-43] There are no traceable objects to add.
# run 1000ns
Hello world! This Is The First Lab Module 5 CINVESTAV
We are Team9
Testing print message1....
Finishing Testing print message...
$finish called at time : 0 fs : File "D:/Vivado/Team9_Lab1_Mod5/dv/tb.sv" Line 73
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:03 ; elapsed = 00:00:08 . Memory (MB): peak = 1280.133 ; gain = 0.000
```

GitHub Repository

https://github.com/CeicGitHub/Fundamentals_Of_Digital_Design_FPGA_5/tree/Lab1_MessageClass

Conclusion / Issues

José Antonio Garcia Madrigal

Personally, I believe that understanding this part of the classes lays the foundation for how to begin implementing initialization methods in System Verilog, how to extend classes by inheriting properties and methods, incorporating attributes, and structuring many concepts to apply them effectively within System Verilog. Especially during this verification stage, it is extremely important to understand how these parameters are implemented and to be able to recognize whether they are being applied correctly.

Alonso Emmanuel Lopez Macias

The use and understanding of classes and constructors are important in the context of HDL, especially when it comes to grasping concepts such as strings and various property methods like inheritance and attributes. This is particularly helpful in the verification phase, especially when designing digital circuits, and even more so when working with a language like System Verilog.

Luis Alberto Castañeda Montaña

Mastering the use of classes and constructors within HDL environments, such as System Verilog, significantly enhances the ability to implement object-oriented concepts like inheritance, encapsulation, and attribute management. This knowledge is crucial not only for handling data types like strings but also for building scalable and reusable verification components, ultimately streamlining the development and debugging of complex digital circuit designs.

Cesar Eduardo Inda Cenicerós

A deep comprehension of class structures and object-oriented principles in System Verilog is fundamental for developing robust and reusable verification environments. Mastery of initialization techniques, inheritance mechanisms, and the integration of custom attributes enables the modular design of verification components and improves code scalability. During the verification phase, the ability to correctly implement and evaluate these constructs is critical, not only to ensure functional correctness but also to maintain maintainability and clarity in complex testbenches. Ultimately, leveraging object-oriented capabilities in System Verilog empowers engineers to architect verification solutions that are both efficient and adaptable to evolving design specifications.